

AI Critique Lab 2

AI Prompt Used:

Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team. We have previous experience with Java and React framework.

Recommended Technology Stack (Entry-Level Friendly)

The proposed technology stack is selected to align with the system requirements, the BCE architectural model used in the design, and the team's prior experience in **Java** and **React**, while keeping complexity manageable for an academic project.

Frontend (Mobile Application)

- **React Native (with Expo)**
 - Leverages existing React knowledge
 - Single codebase for Android and iOS
 - Well-suited for map-centric UI, booking flows, and wallet screens shown in the UI mockups (Sections 3.1.1–3.1.9)
- **TypeScript**
 - Improves code reliability and maintainability for an entry-level team
- **React Navigation**
 - Maps cleanly to the dialog map and use case flows
- **Map SDK**
 - Google Maps API or Mapbox SDK
 - Supports real-time markers, routing, and navigation required in Sections 4.2 and 4.6

Backend

- **Java Spring Boot**
 - Matches team's Java experience
 - Clean separation of Boundary (Controllers), Control (Services), and Entity (JPA Models), consistent with the BCE model described in Section 2.1

- **Spring Data JPA (Hibernate)**
 - Simplifies database interaction for entities such as User, Booking, Wallet, and Transaction
- **Spring Security**
 - Handles authentication, authorization, password hashing, and session/token management
 - Supports security requirements in Section 5.3

Database

- **PostgreSQL**
 - Relational structure fits the detailed data dictionary (Appendix A)
 - Strong support for transactional consistency (wallet, booking, payment)
- **Redis (Optional / Stretch)**
 - For caching real-time availability data (carparks, EV chargers) if scalability is required

External Services & APIs

- **Maps & Routing API**
 - Google Maps Directions API or Mapbox Directions
- **Payment Integration (Mock or Sandbox)**
 - Simulated PayNow QR and Card payment gateway
 - Avoids handling real payment credentials, consistent with constraints in Section 2.5
- **Email Service**
 - SMTP service or SendGrid (sandbox)
 - Used for email verification and password reset links (Section 4.1)

DevOps & Tooling

- **GitHub**
 - Version control and collaboration

- **Postman**
 - API testing aligned with functional requirements
 - **Docker (Optional)**
 - Simplifies deployment consistency but not mandatory for Lab 2
-

Our Response

- The suggested response of a Java Spring Boot backend and a React Native frontend are within the team's capability to do and will serve as a good learning opportunity for most of us to deepen our knowledge in SWE frameworks and languages
- For databases, it recommended us to use PostgreSQL. Many of us have experience with PostgreSQL as we are taking SC2207 Introduction to Databases this semester as well.
- For Devops, it recommended us to use github & docker. All of us have used github for various projects. As for docker, it will be a tool that our group is keen to learn and utilise it for our project.